

팍리스 실무형 일경험 프로그램

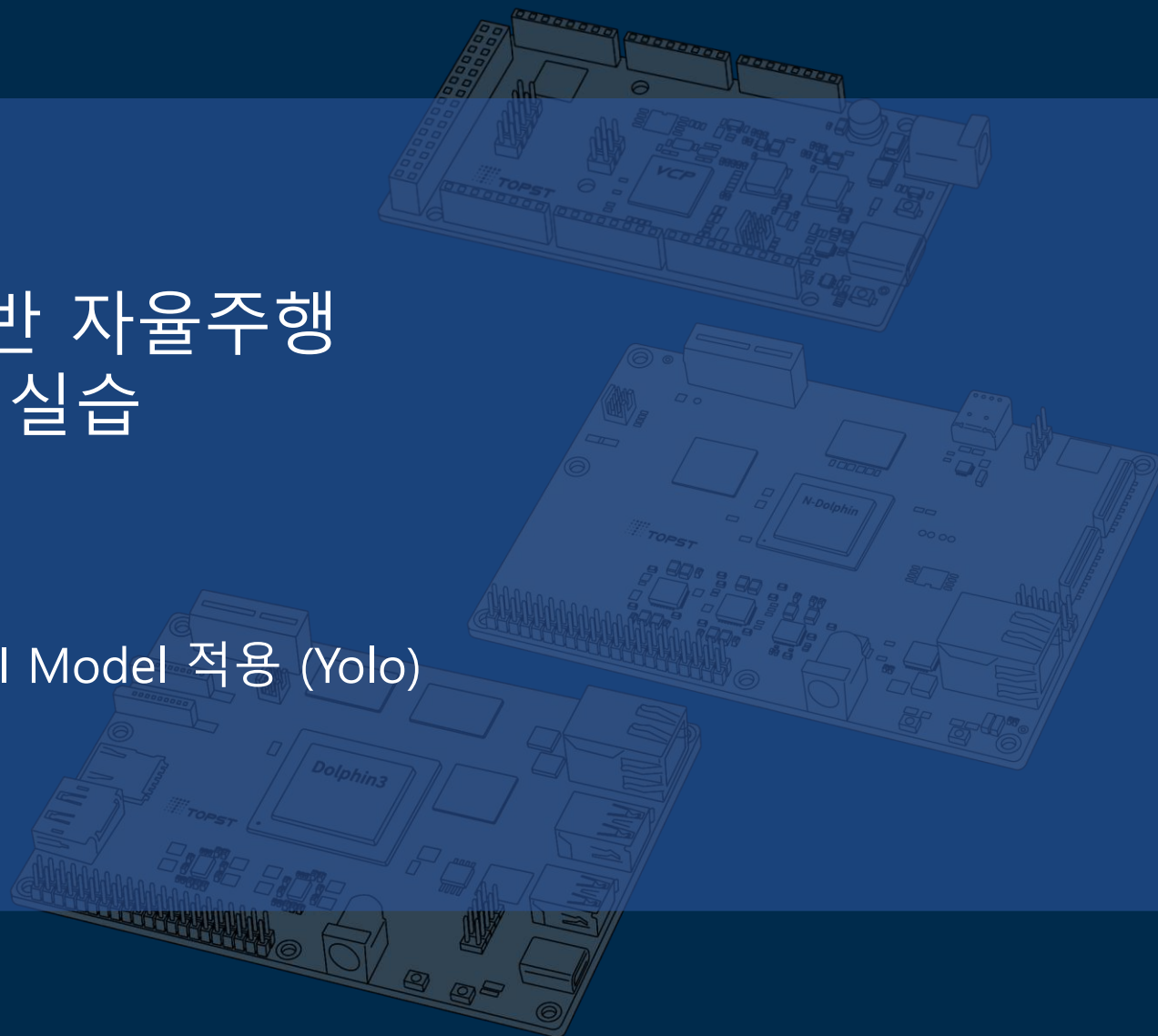
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



6일차

06 Sensing Zone – AI Model 적용 (Yolo)

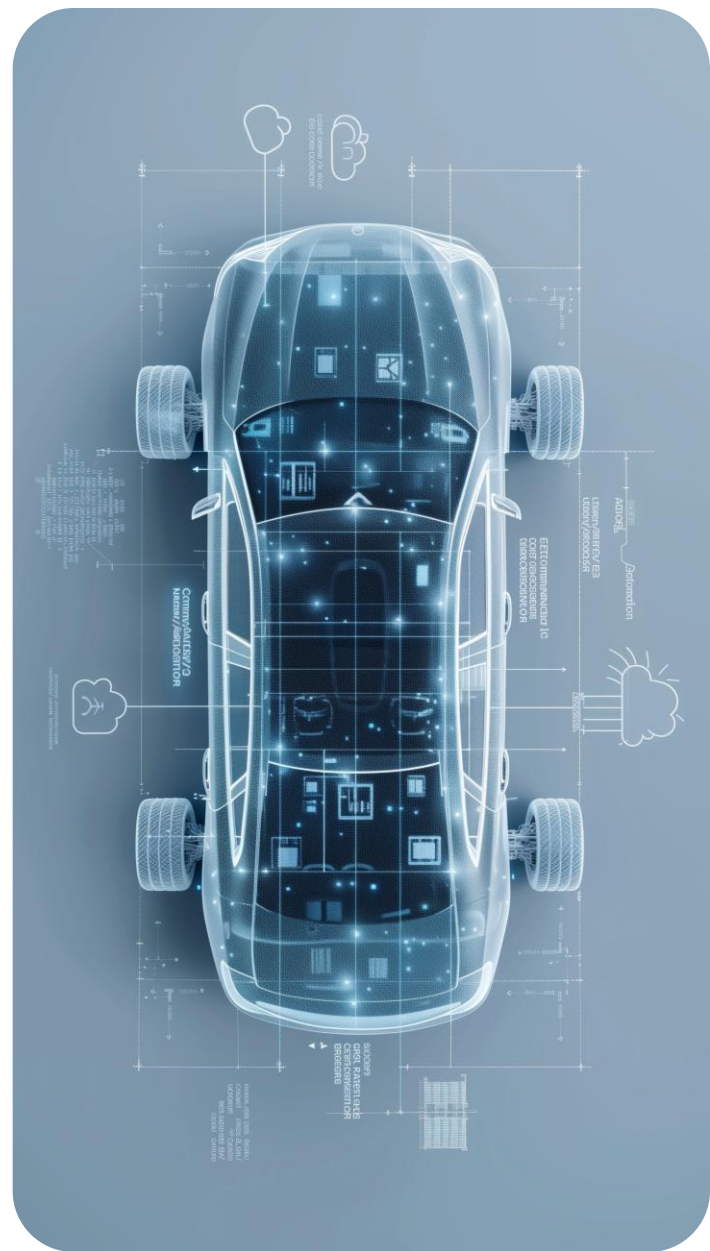
Telechips TOPST



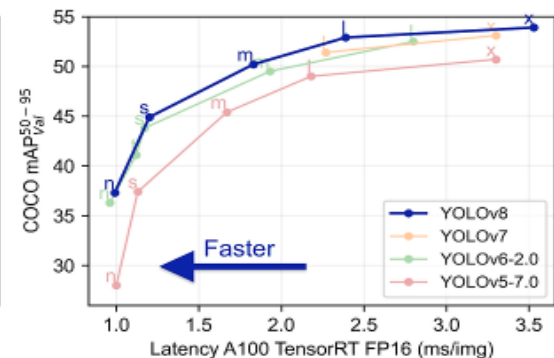
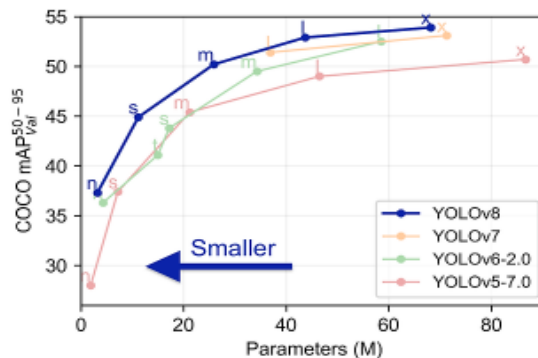
목차

Table of Contents

모델 소개 (yolo)	01
Yolov8s 모델 변환	02
Yolov8s 후처리 과정	03
Yolov8s 모델 추론	04

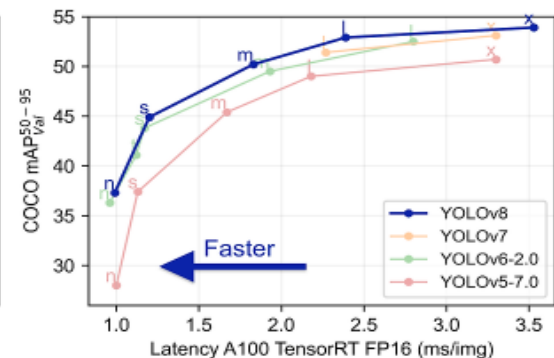
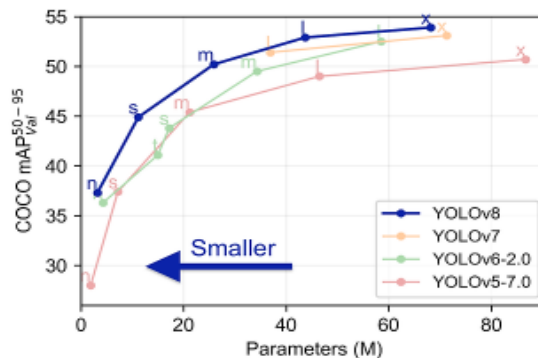


모델 소개-YOLOv8



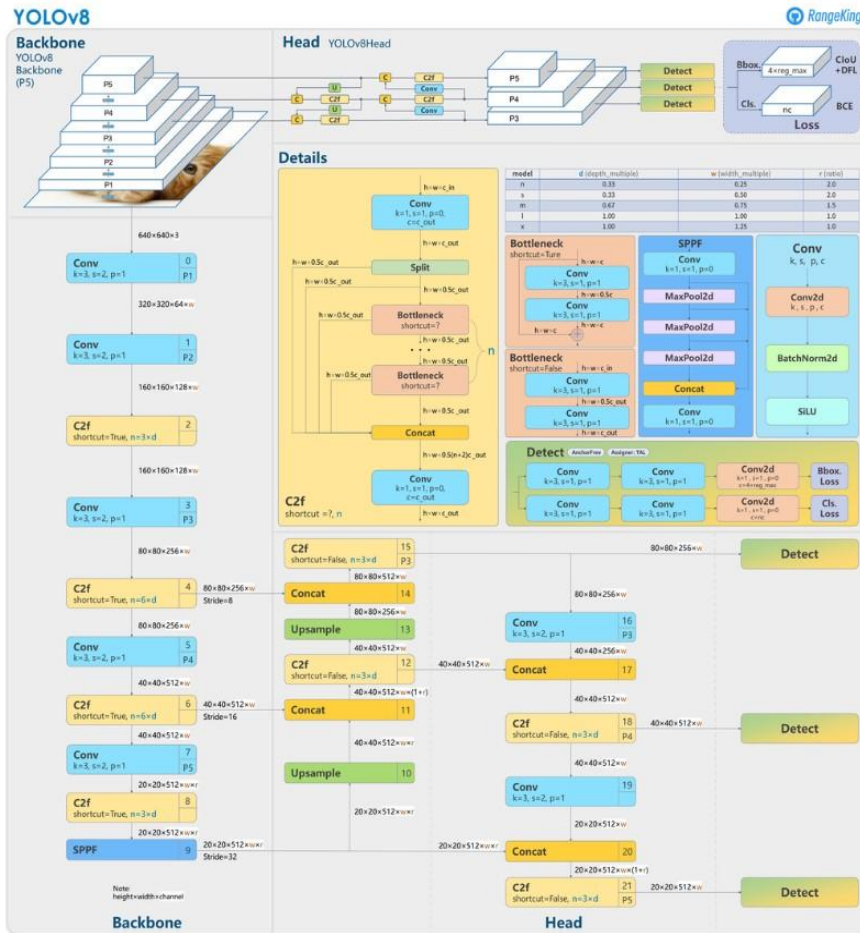
- 객체 감지, 이미지 분류, 인스턴스 Segmentation Task를 사용할 수 있는 통합 YOLO 모델
- YOLOv8은 COCO와 Roboflow 100 데이터 셋에서 이전 모델 보다 더 높은 정확도를 보임
- 모델 크기 n/s/m/l/x로 사용하고자 하는 자원에 맞춰 속도 및 정확도 선택 가능
- ONNX/TFLite/TensorRT 등 배포 경로가 다양하여 임베디드 포팅 용이

모델 소개-YOLOv8



- 주요 특징
 - Anchor-Free 구조 : 기존 YOLOv5 대비 경량화 + 빠른 학습 및 추론
 - Decoupled Head : 분류와 회귀를 분리하여 정확도 개선

모델 소개-YOLOv8

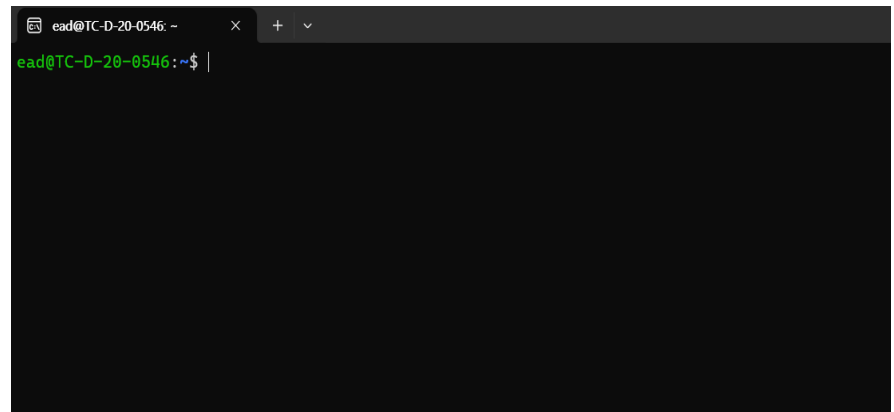
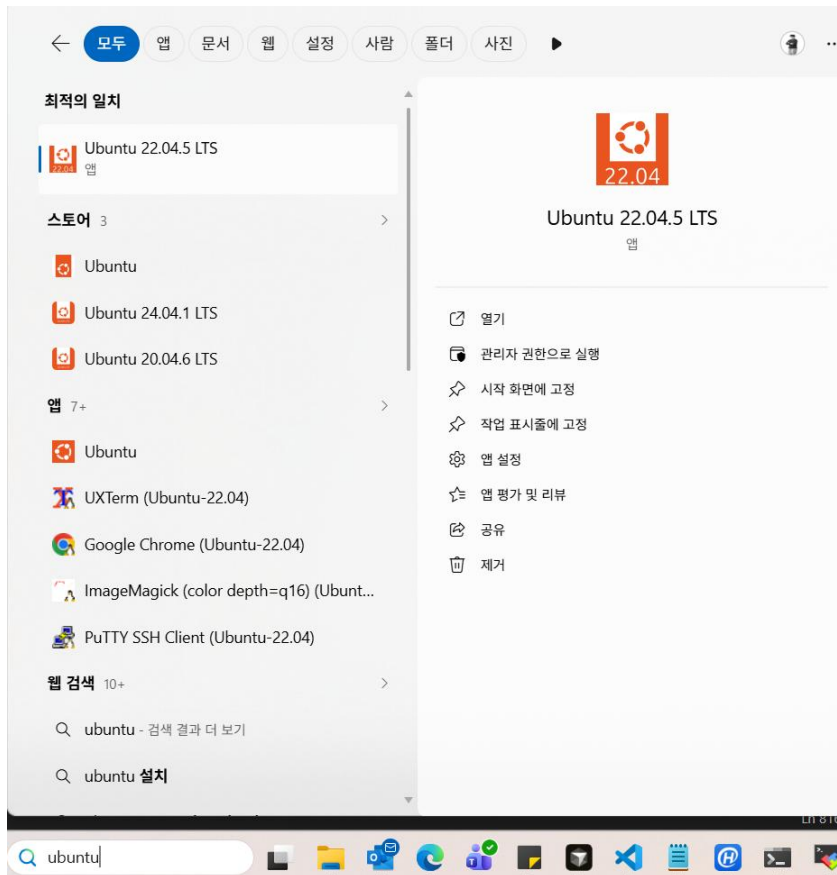


YOLOv8 Architecture, visualisation made by GitHub user RangeKing

- 전체 구조 개요
 - Backbone : 입력 이미지에서 특징 추출
 - Neck : 멀티 스케일 피쳐 통합
 - Head : 객체 예측
- 구조적 개선 포인트
 - C2f 모듈 : 기존 C3 대비 효율 향상
 - DFL : 경계 상자 회귀 정밀도 향상
 - Dynamic Input Shape 지원 : 다양한 해상도 입력에 유연

Yolov8s 모델 변환

- WSL Ubuntu 22.04 실행



Yolov8s 모델 변환

- tc-nn-toolkit 가상 환경 세팅 (wsl ubuntu)

```
$ cd
```

```
$ cd ~/Work/tc-nn-toolkit
```

```
$ . venv/bin/activate
```

- Input 디렉터리에 yolov8s 모델 가져오기

```
$ cd input_networks
```

```
$ git clone https://github.com/topst-development/Education
```

```
$ cd Education
```

```
$ git switch edu/Fabless
```

```
$ ls ai_model/
```

```
$ cd ../../
```

```
(venv) ead@TC-D-20-0546:~/project/tc-nn-toolkit/input_networks/Education$ ls ai_model/  
yolov8s.bin  yolov8s_extracted.onnx
```

Yolov8s 모델 변환

- 모델 컨버팅 수행

```
$ python ./EnlightSDK/converter.py ./input_networks/Education/ai_model/yolov8s_extracted.onnx  
--type obj \  
--add-detection-post-process ./input_networks/Education/ai_model/yolov8s.bin \  
--dataset Custom \  
--dataset-root my_dataset_path \  
--output ./output_networks/yolov8s.enlight \  
--enable-track \  
--mean 0 0 0 \  
--std 1 1 1 \  
--num-class 80 \  
--yolo-version v8 \  
--enable-letterbox \  
--dfl-reg-max 16 \  
--output-order cl --num-images 1
```


Yolov8s 모델 변환

- 모델 컨버팅 수행

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/converter.py \  
./input_networks/Education/ai_model/yolov8s_extracted.onnx \  
--type obj \  
--add-detection-post-process ./input_networks/Education/ai_model/yolov8s.bin \  
--dataset Custom \  
--dataset-root my_dataset_path \  
--output ./output_networks/yolov8s.enlight \  
--enable-track \  
--mean 0 0 0 \  
--std 1 1 1 \  
--num-class 80 \  
--yolo-version v8 \  
--enable-letterbox \  
--dfl-reg-max 16 \  
--output-order cl
```

- type : 네트워크 모델의 후처리 타입 결정 (obj, cls, unknown)
- add-detection-post-process : yolo 모델의 앵커 박스 적용 옵션
- dataset : 양자화 스케일 수집을 위한 데이터셋 타입
- dataset : 양자화 스케일 수집에 사용할 입력 데이터 경로
- output : 모델 변환 결과 저장 위치
- mean : 입력 데이터 정규화 파라미터 (채널별 평균값)

Yolov8s 모델 변환

- 모델 컨버팅 수행

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/converter.py \  
./input_networks/Education/ai_model/yolov8s_extracted.onnx \  
--type obj \  
--add-detection-post-process ./input_networks/Education/ai_model/yolov8s.bin \  
--dataset Custom \  
--dataset-root my_dataset_path \  
--output ./output_networks/yolov8s.enlight \  
--enable-track \  
--mean 0 0 0 \  
--std 1 1 1 \  
--num-class 80 \  
--yolo-version v8 \  
--enable-letterbox \  
--dfl-reg-max 16 \  
--output-order cl
```

- std : 입력 데이터 정규화 파라미터 (채널 별 표준편차)
- num-class : 모델 class 수
- yolo-version : yolo 모델의 버전 설정
- enable-letterbox : letterbox 옵션 설정 여부
- dfl-reg-max : yolo의 dfl 레이어 좌표 분포 수
- output-order : 모델 출력 텐서 순서 -> cl : [N, H, W, C]

Yolov8s 모델 변환

- 모델 양자화 수행

```
$ python ./EnlightSDK/quantizer.py ./output_networks/yolov8s.enlight --  
output ./output_networks/yolov8s_quantized.enlight
```

```
Start FuseConstantEltwLayer optimizer  
debugizing 127 / 219  
optimizing ..... done.  
Start TreatIntermediateOutput optimizer  
debugizing 127 / 219  
optimizing ..... done.  
Start ModifyLayerQuantizationFriendly optimizer  
debugizing 142 / 250  
optimizing ..... done.  
quantization..... done.  
End sanity check for custom quantization  
Start MakeCompilerFriendly optimizer  
debugizing 127 / 252  
optimizing ..... done.  
Start MakeLayerAligned optimizer  
debugizing 127 / 243  
optimizing ..... done.  
Serializing Graph done.  
Writing to File..... Done  
Writing File path : ./output_networks/yolov8s_quantized.enlight  
  
Quantizer done.
```

Yolov8s 모델 변환

- 모델 컴파일 수행

```
$ python ./EnlightSDK/compiler.py ./output_networks/yolov8s_quantized.enlight --th-iou 0.5 --th-conf 0.5
```

```
Start MakeCode(version=NPUV20) code-emitting
code-emitting ..... done.
                ./output_code/yolov8s_quantized/npu_cmd.h created
                ./output_code/yolov8s_quantized/npu_cmd.bin created
                ./output_code/yolov8s_quantized/npu_cmd.txt created
Writing [11196416]/[11196416]
                ./output_code/yolov8s_quantized/quantized_network.bin created
                ./output_code/yolov8s_quantized/oact_entry_table.py created
Notification!!!
Conv2d_42___model_22_cv3_0_cv3_0_2_Conv, Conv2d_53___model_22_cv3_1_cv3_1_2_Conv, Conv2d_62___model_22_cv3_2_cv3_2_2_Conv assigned to conf layers
Conv2d_41___model_22_cv2_0_cv2_0_2_Conv, Conv2d_52___model_22_cv2_1_cv2_1_2_Conv, Conv2d_61___model_22_cv2_2_cv2_2_2_Conv assigned to loc layers
                ./output_code/yolov8s_quantized/network.h created
                ./output_code/yolov8s_quantized/post_process.c created
                ./output_code/yolov8s_quantized/oe_network.bin created
                ./output_code/yolov8s_quantized/oe_network.h created

                NPU compiler: done sucessfully.

Compiler. done.
```

Yolov8s 모델 변환

- 모델 시뮬레이터
 - 목적 : 컨버팅(Enlight 변환)된 모델과 양자화(Quantization)된 모델의 **정확도 및 추론 결과 차이를 비교·검증**하기 위한 오프라인 시뮬레이터.
 - 기능 :
 - 동일한 입력 이미지에 대해
 - FP32 (컨버팅된 모델)
 - INT8 (양자화된 모델)의 출력 결과를 비교
 - Map, IoU, confidence score 등의 지표를 시각적으로 검증
 - 실제 npu 추론 전 단계에서 모델 손실 및 성능 저하율을 사전 점검

Yolov8s 모델 변환

- FP32 모델 시뮬레이터

```
$ python ./EnlightSDK/enlight_sim.py output_networks/yolov8s.enlight --inputs  
sample/dog.png \  
--th-iou 0.5 \  
--th-conf 0.5 \  
--enable-letterbox \  
--dump-shape pytorch \  
--dump-format pytorch
```

Yolov8s 모델 변환

• FP32 모델 시뮬레이터

```

model                : output_networks/yolov8s.enlight
inputs               : sample/dog.png
model_config         : auto
output               : None
result_root          : ./output_result
th_iou                : 0.5
th_conf               : 0.5
has_background       : False
topk                  : 5
force_resize         : None
crop                 : None
use_cv2               : False
enable_letterbox      : True
image_format         : RGB
dump                 : True
dump_root             : ./output_dump
dump_shape            : pytorch
dump_format           : pytorch
enable_show           : False
enable_customize_post_process : None
enable_blazeface_post_process : None
enable_save_npy_result_for_unknown : False
enable_opts           : False

```

Initializing Network done.
Custom_100

Checking arguments...
Save results (result) path : ./output_result/yolov8s/dog

Save results (dump) path : ./output_dump/yolov8s/dog

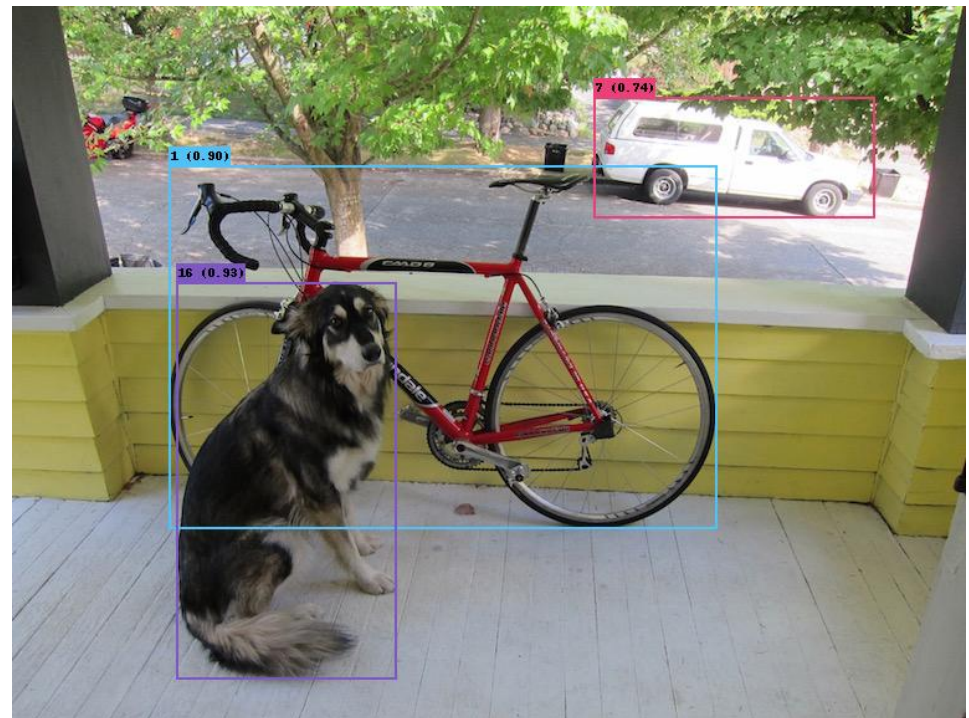
Image resolution: 768x576 (original image resolution)

idx	x1	y1	x2	y2	score(%)	class_number	label
0	126.1368	128.3789	567.6565	420.8864	90.3953	1	1
1	468.0919	73.9053	694.1356	170.8539	74.2017	7	7
2	132.3539	222.7666	309.8087	541.1525	93.4649	16	16

Image resolution: 640x640 (network input resolution)

idx	x1	y1	x2	y2	score(%)	class_number	label
0	105.1140	186.9824	473.0471	430.7387	90.3953	1	1
1	390.0766	141.5878	578.4464	222.3782	74.2017	7	7
2	110.2950	265.6388	258.1739	530.9604	93.4649	16	16

Save results (text) ./output_result/yolov8s/dog/dog_result.txt
 Save results (image) ./output_result/yolov8s/dog/dog_result.png
 [1 / 1]
 Inference. done.



Yolov8s 모델 변환

- INT8 모델 시뮬레이터

```
$ python ./EnlightSDK/enlight_sim.py output_networks/yolov8s_quantized.enlight  
--inputs sample/dog.png \  
--th-iou 0.5 \  
--th-conf 0.5 \  
--enable-letterbox
```


Yolov8s 모델 변환

• INT8 모델 시뮬레이터

```

model          : output_networks/yolov8s_quantized.enlight
inputs         : sample/dog.png
model_config   : auto
output         : None
result_root    : ./output_result
th_iou         : 0.5
th_conf        : 0.5
has_background : False
topk           : 5
force_resize   : None
crop           : None
use_cv2        : False
enable_letterbox : True
image_format   : RGB
dump           : True
dump_root      : ./output_dump
dump_shape     : enlight
dump_format    : enlight
enable_show    : False
enable_customize_post_process : None
enable_blazeface_post_process : None
enable_save_npy_result_for_unknown : False
enable_opts    : False

```

Initializing Network done.
Custom_100

Checking arguments...
Save results (result) path : ./output_result/yolov8s_quantized/dog

Save results (dump) path : ./output_dump/yolov8s_quantized/dog

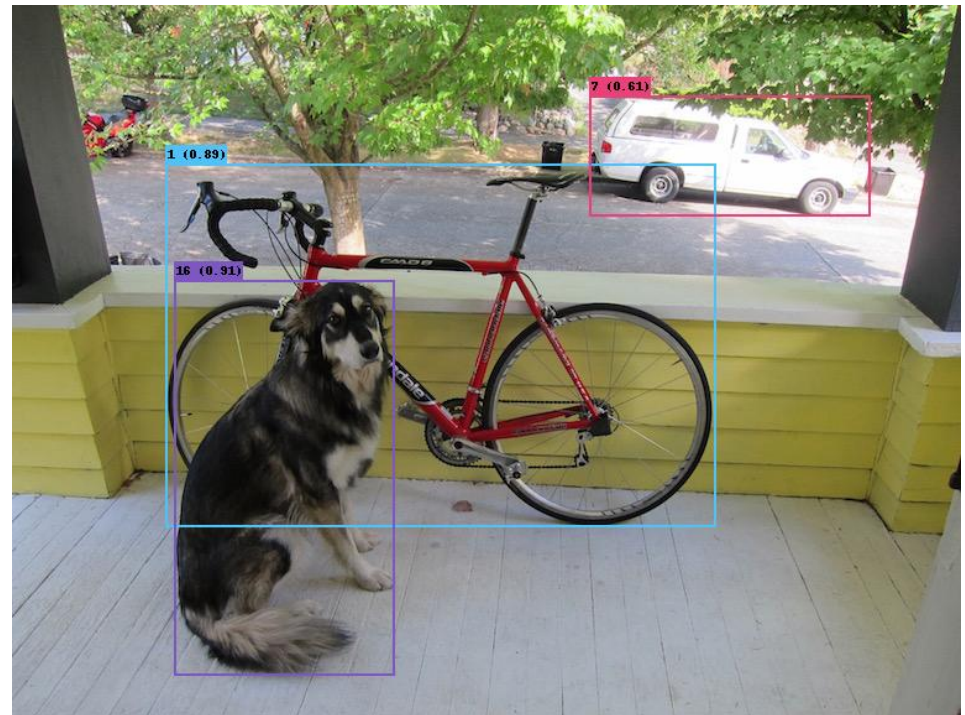
Image resolution: 768x576 (original image resolution)

idx	x1	y1	x2	y2	score(%)	class_number	label
0	124.7915	128.9809	569.1708	421.5896	89.4126	1	1
1	467.2186	73.9025	694.1576	170.9728	60.5090	7	7
2	131.4516	222.8684	309.2574	541.9713	91.2692	16	16

Image resolution: 640x640 (network input resolution)

idx	x1	y1	x2	y2	score(%)	class_number	label
0	103.9929	187.4841	474.3090	431.3247	89.4126	1	1
1	389.3488	141.5854	578.4647	222.4773	60.5090	7	7
2	109.5430	265.7237	257.7145	531.6428	91.2692	16	16

Save results (text) ./output_result/yolov8s_quantized/dog/dog_result.txt
 Save results (image) ./output_result/yolov8s_quantized/dog/dog_result.png
 [1 / 1]
 Inference. done.



Yolov8s 후처리 과정

- 네트워크 및 후처리 빌드 수행

- 모델 복사

- \$ cd build_network

- \$ cp ../output_code/yolov8s_quantized/ ./ -ar

- 링크된 파일 제거

- \$ rm -rf network.h post_process.c

- Yolov8s 네트워크 및 후처리 파일 링크

- \$ ln -s yolov8s_quantized/network.h

- \$ ln -s yolov8s_quantized/post_process.c

- 후처리 빌드 수행

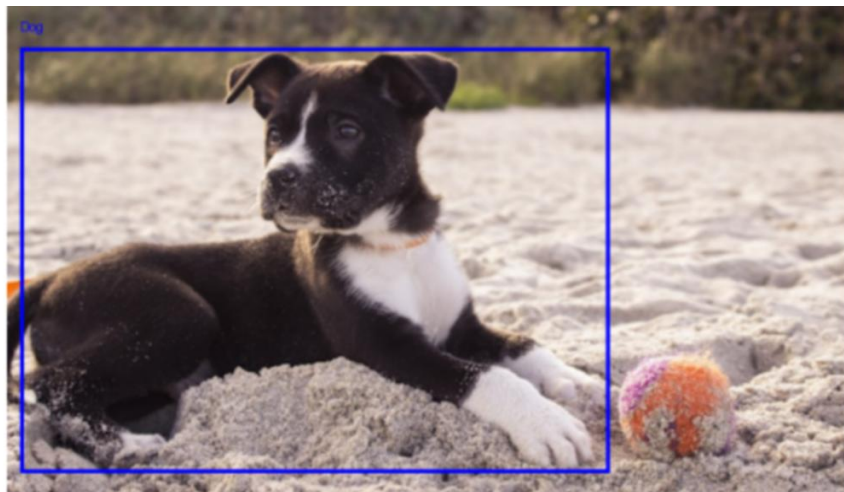
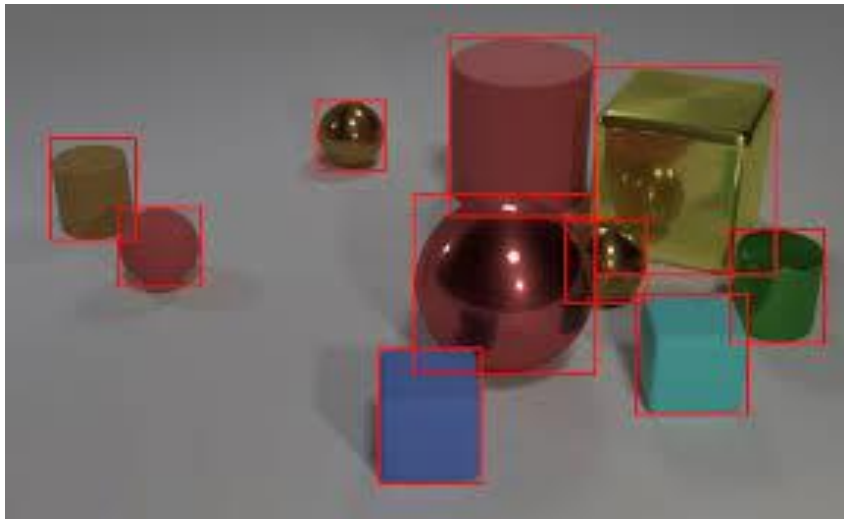
- \$ make

```
aarch64-none-linux-gnu-gcc -shared -Wl,-soname,net.so -o net.so post_process.o detector.o classifier.o yolo_detector.o ssd_detector.o custom_postproc.o enlight_network.o network.o  
rm -rf ./*.o network/*.o enlight/*.o /*.o
```

Yolov8s 후처리 과정

- 네트워크 및 후처리 빌드 수행
 - onnx 모델의 경우 모델 내부에서 후처리를 수행하지 않기 때문에 특정 텐서 형태로 출력됨
 - 해당 출력 텐서를 api 호출을 통해 불러와 추가적인 연산을 진행하여 객체인식을 위한 바운딩 박스 처리를 진행해야함
 - Yolo의 경우 다음과 같은 연산을 수행
 - DFL 연산 (softmax + Expectation)
 - Bounding Box 복원 (Decoding)
 - Sigmoid 연산
 - Confidence 계산
 - Threshold Filtering
 - Non-Maximum Suppression (NMS)
 - Output 출력 (bbox 좌표, score, class, label)

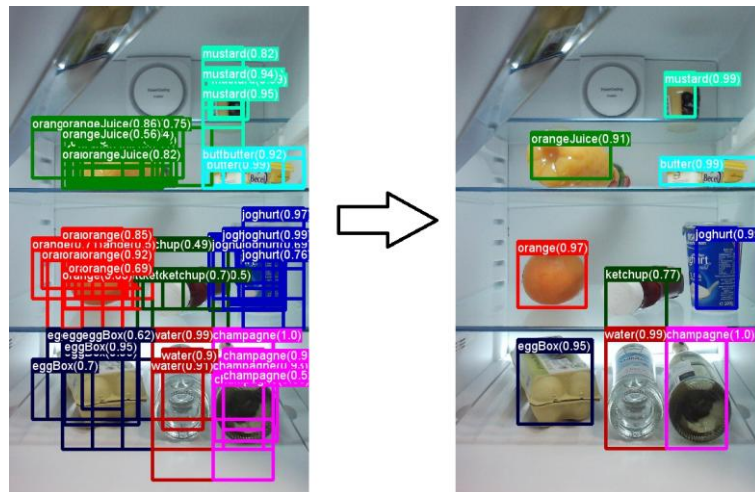
Yolov8s 후처리 과정



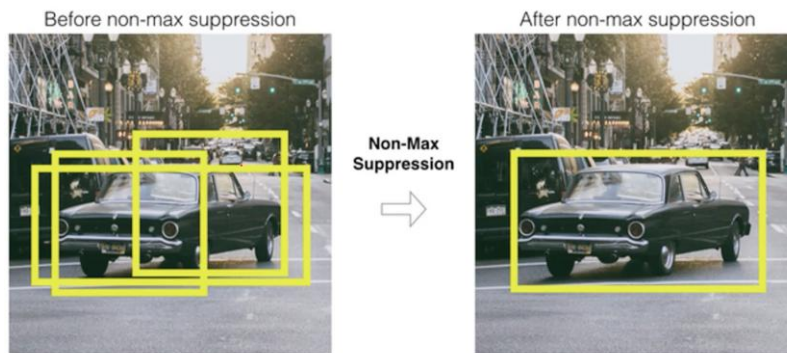
- Bounding Box

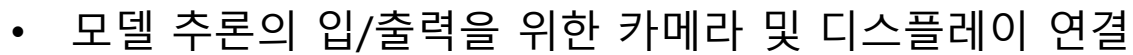
- 이미지 내에서 객체의 위치를 나타내는 직사각형 영역
- Yolo모델은 객체의 위치와 크기를 예측
- 구성 요소
 - x, y 좌표 (중앙점)
 - w, h 값 (너비, 높이)

Yolov8s 후처리 과정

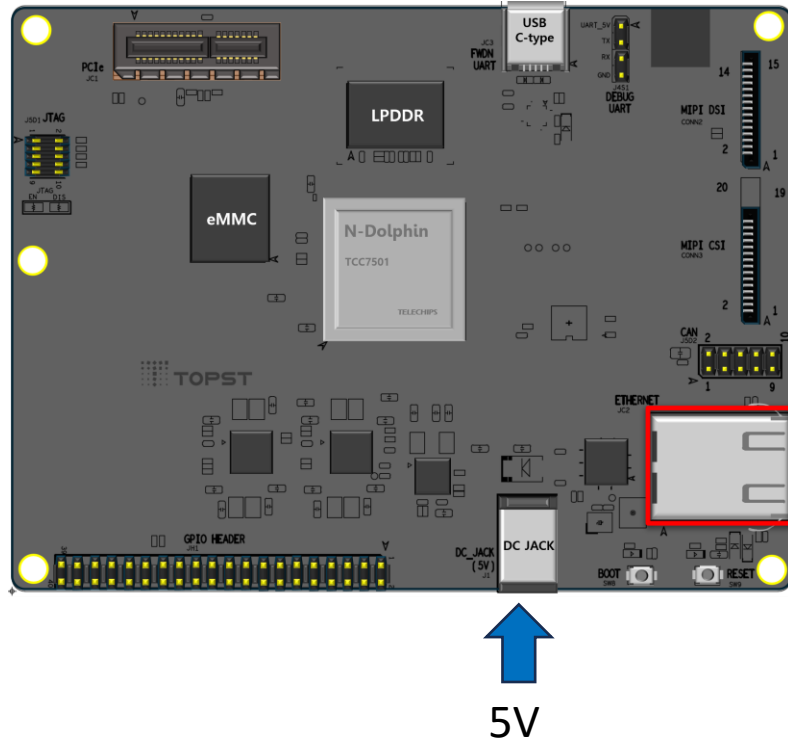


- NMS - 중복 탐지 제거
 - 여러 개의 후보 박스 중 최고 점수를 가진 박스를 선택
 - 중복된 박스는 제거
 - 설정된 임계값 이상인 박스들은 중복으로 간주





모델 추론 – YOLOv8s

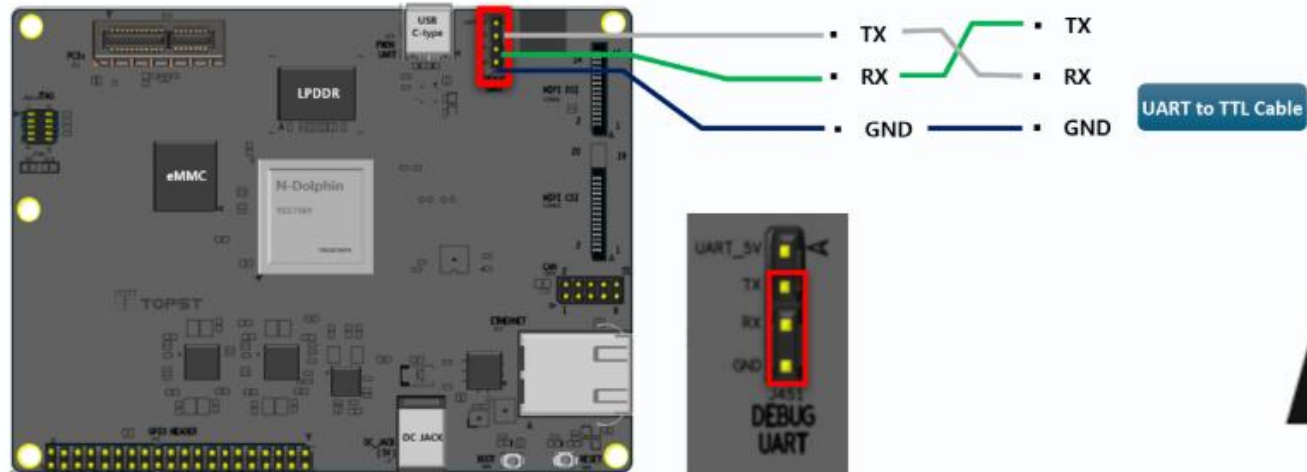


Ethernet Cable



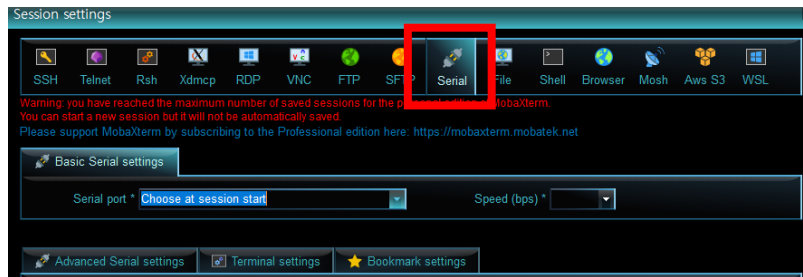
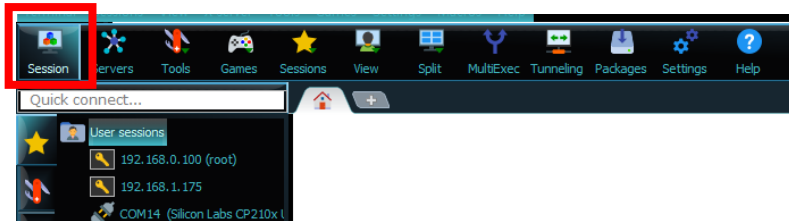
- 이더넷 및 5V 전원 연결

모델 추론 – YOLOv8s

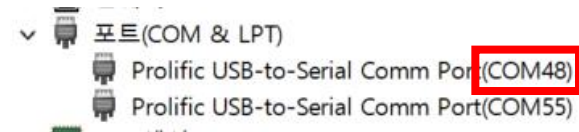


- UART 디버거 연결

모델 추론 – YOLOv8s



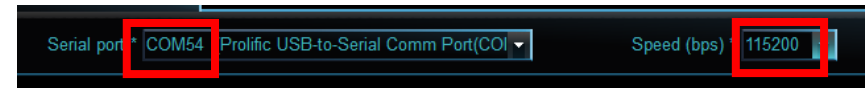
- 윈도우 검색창에 MobaXterm 검색 후 실행
- 오른쪽 상단에 Session 버튼 클릭
- Serial 클릭
- 장치 관리자를 통해 port 확인



모델 추론 – YOLOv8s



- Serial port에서 연결된 포트 설정
- Speed 115200 설정



- Advanced Serial settings 탭 선택
- Flow contro에서 None 선택
- OK 눌러 보드 접속
- ID : root / PW : root 입력

모델 추론 – YOLOv8s

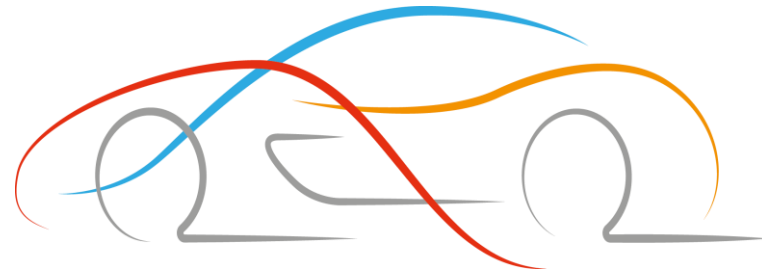
- 네트워크 및 후처리 빌드 수행
 - 후처리 파일 복사 및 AI-G 보드 전송(**ws1 ubuntu** 에서 진행)
\$ cp net.so yolov8s_quantized
\$ scp -r yolov8s_quantized root@192.168.0.100:/home/root/ -> 비밀번호 root 입력

모델 추론 – YOLOv8s

- NPU 추론 수행(보드와 연결된 터미널창에서 수행)
 - CSI CAMERA, DSI DISPLAY 연결 후 보드 부팅
 - 추론 명령어 입력 (보드에서 실행)

```
$ tcnnapp -n yolov8s_quantized -p /dev/video2
```
 - 추론 결과 확인





Thank You